



Ing. Irina Fong



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO
DEPARTAMENTO DE INGENIERÍA DE SOFTWARE
CARRERA INGENIERÍA DE SOFTWARE
INGENIERÍA WEB

Fecha Límite de Entrega:

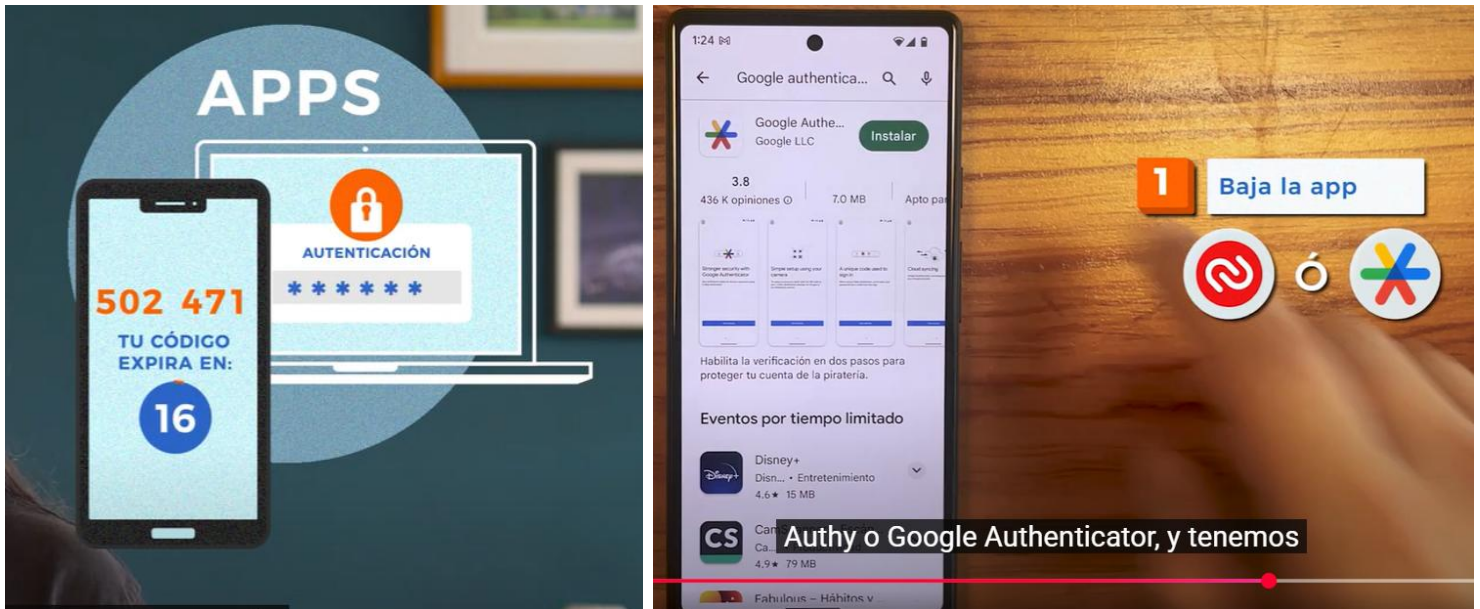
II SEMESTRE 2025

Instructor: Ing. Irina Fong

 **Guía de Laboratorio: Implementación de Autenticación de
Dos Factores (2FA) en PHP**

 Objetivo del Laboratorio

El objetivo de este laboratorio es comprender la importancia de la autenticación de dos factores (2FA) y aprender a implementarla en un sistema de autenticación utilizando PHP, MySQL y la aplicación móvil **Google Authenticator**.



🧠 ¿Cómo Funciona la Autenticación de Dos Factores?

El flujo básico de la autenticación 2FA es el siguiente:

1. **Inicio de sesión:** El usuario ingresa su nombre de usuario y contraseña.
2. **Solicitud de autenticación:** Si las credenciales son correctas, se solicita un segundo factor.
3. **Verificación del segundo factor:** El usuario introduce un código temporal generado por una app (como Google Authenticator).
4. **Acceso concedido:** Si el segundo factor es válido, el usuario accede al sistema.

🧰 Requisitos Previos

Antes de comenzar, asegúrate de tener:



- Un servidor PHP funcional (Apache, Nginx, etc.)
- MySQL instalado y accesible
- Composer instalado para gestión de dependencias PHP
- Aplicación móvil **Google Authenticator** instalada
- Una base de datos con tabla de usuarios con campos adecuados

Paso 1: Configurar la Base de Datos

Ejecuta la siguiente sentencia SQL para agregar el campo secret_2fa a tu tabla de usuarios:

sql

CopiarEditar

```
ALTER TABLE `usuarios`
```

```
ADD `secret_2fa` VARCHAR(255) NULL AFTER `HashMagic`;
```

Paso 2: Formulario de Registro de Usuario

Crea un formulario que contenga los siguientes campos:

- Nombre
- Apellido
- Correo electrónico (como nombre de usuario)
- HashMagic (contraseña encriptada)
- Sexo (M/F)

 El campo secret_2fa **no se llena manualmente**, se genera dinámicamente durante la activación de 2FA.

Paso 3: Instalar Librería Google Authenticator

Desde la raíz de tu proyecto PHP, ejecuta:

```
composer require sonata-project/google-authenticator
```

Integración de Google Authenticator

Google Authenticator genera códigos de tiempo limitado basados en el estándar TOTP (Time-Based One-Time Password). Para implementar esta funcionalidad en PHP, podemos utilizar la biblioteca PHP sonata-project/google-authenticator.

```
composer require sonata-project/google-authenticator
```

Integración de Google Authenticator

Google Authenticator genera códigos de tiempo limitado basados en el estándar TOTP (Time-Based One-Time Password). Para implementar esta funcionalidad en PHP, podemos utilizar la biblioteca PHP sonata-project/google-authenticator.

```
composer require sonata-project/google-authenticator
```

Después de instalar la biblioteca, puedes generar un nuevo "secreto" para cada usuario cuando activan 2FA y almacenarlo en la base de datos.

```
use SonataGoogleAuthenticatorGoogleAuthenticator;

$g = new GoogleAuthenticator();
$usuario_id = obtenerUsuarioId(); // Asegúrate de implementar esta función
$secret = $g->generateSecret();
guardarSecretEnBaseDeDatos($usuario_id, $secret);
```

🔑 Paso 4: Generar el Secreto 2FA y Código QR

En el proceso de activación de 2FA (una vez el usuario ha iniciado sesión), genera un secreto:

php

CopiarEditar

```
use Sonata\GoogleAuthenticator\GoogleAuthenticator;
```



```
use Sonata\GoogleAuthenticator\GoogleQrUrl;
```

```
$g = new GoogleAuthenticator();  
$secret = $g->generateSecret();
```

```
guardarSecretEnBaseDeDatos($usuario_id, $secret); // Implementa esta función
```

```
$correo = 'usuario@ejemplo.com';  
$app = 'MiSistema';  
$url = GoogleQrUrl::generate($correo, $secret, $app);  
$qr_url = 'https://api.qrserver.com/v1/create-qr-code/?size=200x200&data=' . $url;  
  
echo '';
```

Paso 5: Verificar el Código Generado por el Usuario

En la segunda etapa del inicio de sesión (después de usuario y contraseña):

php

CopiarEditar

```
$codigo_usuario = $_POST['codigo_2fa']; // Código ingresado por el usuario
```

```
$secret = obtenerSecretDesdeBaseDeDatos($usuario_id);
```

```
if ($g->checkCode($secret, $codigo_usuario)) {
```

```
    echo "✅ Acceso concedido";
```

```
} else {
```

```
    echo "❌ Código incorrecto";
```

```
}
```

```
generarSecreto.php U Autenticar.php U X ImprimirSecreto.php U ▶
Autenticar.php
1 <?php
2 require 'vendor/autoload.php'; //Librería Composer
3 use Sonata\GoogleAuthenticator\GoogleAuthenticator;
4 use Sonata\GoogleAuthenticator\GoogleQrUrl;
5 $secret = "BHDLLXVOTZZTCE6G";
6
7 // Procesar formulario
8 $mensaje = '';
9 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
10     $codigo = $_POST['codigo'];
11     $g = new GoogleAuthenticator();
12
13     if ($g->checkCode($secret, $codigo)) {
14         $mensaje = "<p style='color: green;'>✅ Código válido.
15         // Aquí puedes guardar en sesión que está verificado
16     } else {
17         $mensaje = "<p style='color: red;'>❌ Código incorrecto
```

Actividades del Laboratorio

1. Modifica el sistema de autenticación existente para incluir soporte para 2FA.
2. Genera y almacena dinámicamente el secret_2fa cuando el usuario habilita 2FA.
3. Muestra un código QR para que el usuario lo escanee con Google Authenticator.
4. Crea una segunda pantalla de verificación para introducir el código temporal.
5. Valida el código temporal antes de permitir el acceso completo.

✓ Criterios de Evaluación

- Implementación correcta del campo secret_2fa en la BD
- Código QR generado dinámicamente
- Secreto guardado por usuario
- Validación exitosa del código TOTP
- Acceso denegado si el código no es válido

Recursos Adicionales

- <https://www.youtube.com/watch?v=jHH8qsTlTgo>
- <https://nelkocodev.com/php/implementando-autenticacion-de-dos-factores-en-php-una-guia-integral/>